

Statically Typed Purely Functional Definitional Interpreters for *Essentials of Programming Languages*

Kwanghoon Choi^[0000–0003–3519–3650] ✉

Chonnam National University, Gwangju 61186, Republic of Korea
`kwanghoon.choi@jnu.ac.kr`

Abstract. Definitional interpreters provide an executable and pedagogically effective way to explain programming language semantics. However, when implemented in a dynamically typed and effectful metalanguage, semantic components such as stores, continuations, handlers, substitutions, and scheduling states may remain implicit in the host-language implementation, making it harder to see how language extensions affect interpreter structure. This paper reconstructs the family of interpreters from *Essentials of Programming Languages* (EOPL) in Haskell, a statically typed and purely functional metalanguage, for pedagogical use. We preserve the object languages, examples, and tests of EOPL while making their semantic components explicit through algebraic data types, evaluator signatures, and explicit semantic domains. Across 15 interpreters and 6 type checkers, language extensions are exposed as systematic type-directed changes.

The reconstruction is currently used in undergraduate and graduate programming language courses, where students learn interpreter construction through incremental and test-driven development. We further illustrate its pedagogical use through three case studies that progressively extend students’ experience: composing mutable state and exception handling, deriving an actor-based interpreter from the typed structure of the EOPL thread interpreter, and translating the CHECKED interpreter and type checker into Rocq to mechanize type safety. Together, the classroom experience and case studies suggest that the reconstruction provides a concrete bridge from learning programming language concepts through executable interpreters to language design and mechanized reasoning.

Keywords: Definitional Interpreters · Static Types · Semantics · Haskell · Programming Language Education

1 Introduction

Programming language textbooks written so far can be broadly categorized into two groups. The first group focuses on the taxonomy of programming languages, classifying programming paradigms such as functional languages (e.g., Scheme, ML, Haskell), object-oriented languages (e.g., C++, Java, Python), and logic

Table 1. A Brief History of Semantics-based Programming Language Textbooks

Title of Programming Language Textbooks (Year)	Languages
The Formal Semantics of Programming Languages (1993)	Math
Theories of Programming Languages (1998)	Math
Types and Programming Languages (2002)	Math & OCaml
Semantics with Applications: An Appetizer (1992-2007)	Math
Structure and Interpretation of Computer Programs (1985-2022)	Scheme/JavaScript
Essentials of Programming Languages (1983-2008)	Scheme
Programming Languages: Appl. & Interpret. (2000-2023)	Scheme
Software Foundations (2011–2026)	Rocq
Programming Language Foundations in Agda (2019–2026)	Agda

languages (e.g., Prolog), and introducing the key features of representative programming languages in each paradigm.

The second group deals with programming language semantics, focusing on rigorously defining and understanding the behavior of programs. It aims to provide the basic mathematical techniques necessary for those who are beginning a study of the semantics and logics of programming languages [1]. Some textbooks in the group are shown in Table 1. The table highlights how education in the second group is shifting to emphasize not only the mathematical techniques for defining semantics but also the approach of explaining programming languages through *interpreters*.

A more recent direction in this second group is to use interactive theorem provers as textbook languages. Textbooks such as *Software Foundations* [2] and *Programming Language Foundations in Agda* [3] present programming language semantics not only as mathematical definitions or executable interpreters, but also as mechanized formal developments. In this approach, syntax, operational semantics, type systems, and meta-theoretic properties such as progress and preservation are encoded directly in a proof assistant, and their proofs are checked mechanically. Thus, this line of textbooks can be seen as combining the mathematical semantics tradition with executable and machine-checkable artifacts.

We favor this interpreter-based approach because providing a semantics model that can be directly executed through a working program makes it easier for students to engage with and learn the material. Furthermore, directly modifying this program can facilitate the design and exploration of new programming languages, making the process more intuitive and enjoyable through hands-on experimentation. A textbook such as *Essentials of Programming Languages (EOPL)* [4] demonstrates this approach particularly well: a family of small languages is introduced incrementally, and each new language feature is explained by extending an interpreter for a previous language as shown in Figure 1

Moreover, EOPL covers, within a single incremental interpreter-based framework, a particularly broad spectrum of programming language features, including the lambda calculus, recursion, explicit and implicit states, continuations, excep-

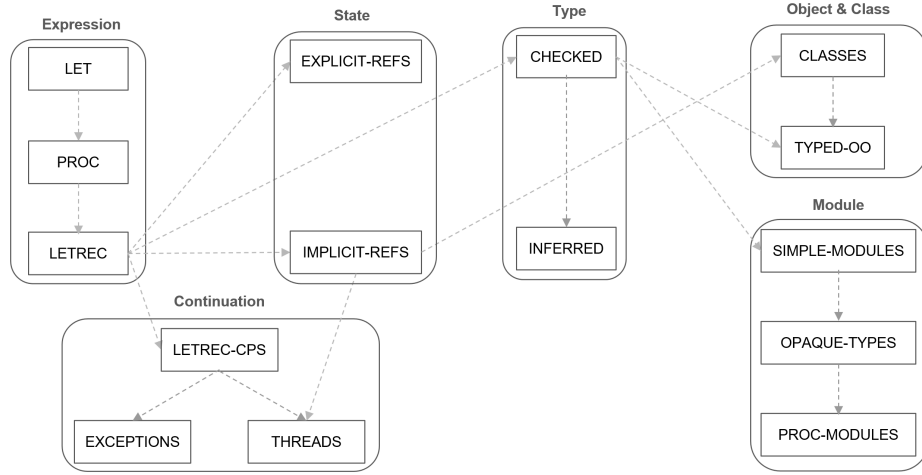


Fig. 1. Each box denotes a programming language in EOPL, and an arrow from language A to language B indicates that B is obtained by adding new features to A. Table 2 summarizes the main feature of each language.

tions, concurrency, type checking, type inference, modules & interfaces, opaque types, module procedures, and classes & objects, as summarized in Table 2. To the best of our knowledge, we have not found another textbook or related work that covers this breadth of language features within a single incremental interpreter-based framework.

The interpreter-based approach requires selecting a meta-language for concrete implementation. EOPL asserts that any language supporting both first-class procedures and assignments (ML, Common Lisp, Python, Ruby, etc.) is suitable for implementing interpreters. Among them, it chooses Scheme—a dynamically typed functional language with support for assignments [4].

However, the essence of EOPL could be effectively taught using types, making the explanation of interpreters more direct and intuitive. A well-known advantage of static types is that function type signatures effectively describe the function’s behavior. Static types for interpreter functions can also highlight similarities and differences among interpreters in a systematic way. In fact, the third edition of EOPL newly incorporates informal type declarations, called contracts, in nontrivial function definitions to clarify key implementation ideas, which aligns well with our motivation. However, these contracts do not fully capture all essential aspects of the implementations, leaving certain details implicit. Then, a thought arose: why not use a statically typed functional language for EOPL instead, making these aspects explicit?

Adopting a purely functional language enhanced our strategy of using static types in EOPL as every side effect such as state and error handling in the interpreters must be explicitly reflected in their function type signatures, which would otherwise remain hidden under implicit effects.

Table 2. The Status of the 15 Programming Languages in EOPL (IN: Interpreter, TC: Type Checker)

Topics/PLs	IN	TC	Description
Expression			
LET	✓	n/a	Lexical scope
PROC	✓	n/a	The λ -calculus
LETREC	✓	n/a	Recursive Procedure
State			
EXPLICIT-REFS	✓	n/a	C-style Pointer
IMPLICIT-REFS	✓	n/a	Java-style Reference
Continuation			
LETREC-CPS	✓	n/a	Continuation-Passing Style
EXCEPTIONS	✓	n/a	Exception Handling
THREADS	✓	n/a	Multi-threads and Mutexes
Types			
CHECKED	✓	✓	Type Checking
INFERRED	✓	✓	Type Inference
Modules			
SIMPLE-MODULES	✓	✓	Simple Modules and Interfaces
OPAQUE-TYPES	✓	✓	Opaque and Transparent types
PROC-MODULES	✓	✓	Module Procedures
Objects & Classes			
CLASSES	✓	n/a	Class, Object, Interface, Inheritance
TYPED-OO	✓	✓	Type Checking OO-PL

Our goal is to develop statically typed and purely functional EOPL-style definitional interpreters. We do not propose new object languages, nor do we seek to replace the original textbook semantics. Instead, we intentionally preserve the object languages, examples, and tests of EOPL as a reference point, while reconstructing their implementation architecture in Haskell. This allows us to separate two concerns: the object-language behavior remains close to the original EOPL interpreters, whereas the metalanguage representation exposes the semantic components that were previously implicit.

The reconstruction is currently used in two programming language courses at the author's university, targeting different academic levels. The earlier part of EOPL has been taught to third-year undergraduate students, while the later part has been taught to first-year master's students. The corresponding lecture materials and Haskell source code are publicly available¹.

To further illustrate its pedagogical use, we present three case studies that expose students to progressively broader uses of typed definitional interpreters: combining mutable state and exception handling demonstrates how indepen-

¹ https://github.com/kwanghoon/Lecture_EOPL

dently introduced language features can be composed; deriving an actor-based interpreter from the typed structure of **THREADS** demonstrates how an existing interpreter can guide the design of a new language; and translating the **CHECKED** interpreter and type checker into Rocq demonstrates how executable interpreters can serve as a starting point for mechanized reasoning. Together, the classroom use and these case studies show how the reconstruction can support a progression from understanding language features, through interpreter extension and language design, to formal verification.

In summary, this paper contributes a statically typed and purely functional reconstruction of the EOPL interpreter family, together with a typed design space that makes the semantic structure of language extensions explicit. The reconstruction is intended for pedagogical use and is supported by classroom experience, accompanying teaching materials, and case studies on feature composition, language extension, and mechanized reasoning.

Section 2 reviews EOPL. Section 3 then presents a typed design space and a type-directed reconstruction of EOPL in a pure metalanguage. Section 4 presents three case studies that further illustrate the pedagogical use of our EOPL reconstruction. Section 5 discusses our teaching experience with the reconstruction and related work, and Section 6 concludes.

2 Background: EOPL-style interpreters and type checkers

The *Essentials of Programming Languages (EOPL)* is a well-known textbook that presents a family of small programming languages organized around six major topics: expressions, state, continuations, types, modules, and objects&classes. As shown in Figure 1, the textbook introduces 15 languages incrementally: each box denotes a programming language, and an arrow from language A to language B indicates that B is obtained by adding new features to A.

Table 2 summarizes the main feature of each language. The expression languages **LET**, **PROC**, and **LETREC** introduce the basic structure of definitional interpreters for lexical binding, procedures, and recursive procedures. The state languages explain two styles of mutable state: **EXPLICIT-REFS** models C-style references, where locations are explicitly manipulated by programs, whereas **IMPLICIT-REFS** models Java-style references, where locations are used implicitly. The continuation languages introduce continuations as explicit representations of control contexts and use them to explain continuation-passing style, exception handling, and threads. The typed languages present both type checking and type inference. The module languages introduce mechanisms for programming in the large, and the object&class languages present object-oriented features as an alternative form of program organization.

In EOPL, all 15 languages are presented through executable interpreters, and the later languages in the topics of types, modules, and objects&classes are also accompanied by type checkers. In our reconstruction, we implemented these interpreters and type checkers as working Haskell programs. The purpose of this reconstruction is not to introduce new object languages, but to expose

the implementation architecture of the EOPL interpreter family in a statically typed and purely functional metalanguage. In the following sections, we use this artifact to analyze how language extensions correspond to systematic changes in evaluator signatures, semantic domains, and explicit runtime components.

3 A Typed Design Space and Type-Directed Reconstruction of EOPL

3.1 Running Example: LET

Before discussing the full typed design space, we introduce the basic semantic domains used throughout this paper with a small example from the LET language in Figure 2. Consider the following LET program:

```
let x = 3 in -(x,1)
```

This program is written in the concrete syntax of the object language. After parsing, it is no longer treated as a string, but as an abstract syntax tree. In our Haskell representation, it is represented by an expression of type `Exp`, for example:

```
Let_Exp "x" (Const_Exp 3) (Diff_Exp (Var_Exp "x") (Const_Exp 1))
```

Thus, `Exp` represents the abstract syntax of object-language programs. It is not the result of executing a program; rather, it describes the syntactic structure of a program phrase before evaluation.

Evaluation takes place relative to an environment. In the example above, the body `-(x,1)` must be evaluated under the information that `x` is bound to the value of 3. This information is represented by an environment of type `Env`, such as:

```
Extend_env "x" (Num_Val 3) Empty_env
```

An environment is therefore a semantic context that associates variable names with their meanings. In the basic LET interpreter, this meaning is an expressed value. The evaluator consequently has the type:

```
value_of :: Exp -> Env -> ExpVal
```

This type can be read as follows: given an expression and an environment, the interpreter produces an expressed value.

The result of evaluating the example program is the integer value 2. However, in the interpreter, this result is not represented directly as a Haskell integer of type `Int`. Instead, it is represented as an object-language value:

```
Num_Val 2
```

The type of this value is `ExpVal`. Thus, `ExpVal` represents the runtime values of the object language. In LET, expressed values include numbers and booleans.

As the language is extended, the definition of `ExpVal` is also extended: procedure values are added in `PROC` and `LETREC`, reference values are added in `EXPLICIT-REFS`, mutexes and queues are added in `THREADS`, and object values are added in `CLASSES`.

For typed languages, one more distinction is necessary as follows. The expression `Const_Exp 3` is an abstract syntax tree of type `Exp`; when evaluated, it produces the expressed value `Num_Val 3` of type `ExpVal`; when statically analyzed, it is assigned the object-language type `TyInt` of type `Type`. These three levels should not be confused:

`Const_Exp 3 :: Exp      Num_Val 3 :: ExpVal      TyInt :: Type`

The first is a syntactic representation of a program phrase, the second is a runtime result, and the third is a static classification.

This simple example explains the core domains that appear repeatedly in the rest of the paper. The basic evaluator for `LET` has the form `Exp -> Env -> ExpVal`. Later language features modify this structure in systematic ways. Mutable state adds an explicit `Store`; control features add continuations of type `Cont`; concurrency adds scheduling state; and type checking and type inference introduce type environments, types, substitutions, and error effects. The main claim of this paper is that these changes become visible and systematic when EOPL-style interpreters are reconstructed in a statically typed and purely functional metalanguage.

```

data Exp =
    Const_Exp Int
  | Diff_Exp Exp Exp
  | IsZero_Exp Exp
  | If_Exp Exp Exp Exp
  | Var_Exp Ident
  | Let_Exp Ident Exp Exp

data ExpVal =
    Num_Val Int
  | Bool_Val Bool

data Env = Empty_env
  | Extend_env Ident ExpVal Env
    
```

Fig. 2. Abstract Syntax Trees (`Exp`), Expressed Values (`ExpVal`), and Environments (`Env`) for `LET`, where `Ident` a type synonym for `String`

3.2 A Reconstruction of the EOPL interpreters and type checkers

We now explain how static types help structure and clarify the elements of programming languages in our reconstruction of EOPL. Algebraic data types provide a natural way to represent programming language constructs by composing sum types, product types, and recursive types [5]. This allows language elements to be expressed as mathematical domains (*inductive sets*), where each element—such as expressions (`Exp`), environments (`Env`), continuations (`Cont`),

Table 3. Main axes of the typed design space

Axis	Meaning	Example
Exp	Abstract syntax tree	Const_Exp 3, Var_Exp "x", Let_Exp ...
Env	Variable/recursive bindings	$x \mapsto \text{Num_Val } 3$
ExpVal	Semantic values	Num_Val 3, Bool_Val True, Proc_Val ...
Store	Mutable state	$\text{location} \mapsto \text{ExpVal}$
Cont	Control context	Diff1_Cont exp2 env cont, End_Cont
Type	Types, interfaces, classes	TyInt, TyBool, TyFun ty1 ty2, TyClass Ident

stores (**Store**), expressed values (**ExpVal**), and types (**Type**)—is represented by a corresponding algebraic data type, as summarized in Table 3.

Static types are useful not only for representing these domains, but also for distinguishing elements that may look similar at the surface level. For example, a number in a program is initially represented as the string "123" of type **String**. After parsing, it is transformed into an abstract syntax tree node such as **Const_Exp 123** of type **Exp**. An interpreter then evaluates this tree to produce the expressed value **Num_Val 123** of type **ExpVal**. Finally, in a typed language, the static type of this expressed value is represented as **TyInt** of type **Type**. Thus, static types make explicit the different stages through which a program phrase passes: concrete syntax, abstract syntax, runtime value, and static type.

Beyond distinguishing individual language components, static types also reveal how the architecture of an interpreter changes as a language is extended. We therefore characterize the reconstruction of EOPL interpreters as a set of recurring type-directed extension patterns. Some language extensions preserve the overall evaluator type and only extend the syntactic and semantic domains, as in pure expression extensions that add new forms to **Exp** and new values to **ExpVal**. Other extensions require changes to specific semantic components: binding constructs change the structure of **Env**, mutable state turns a direct evaluator of type **Exp** $\rightarrow$ **Env** $\rightarrow$ **ExpVal** into a store-passing evaluator of type **Exp** $\rightarrow$ **Env** $\rightarrow$ **Store** $\rightarrow$ (**ExpVal**, **Store**), and control features introduce continuations by changing the evaluator into a continuation-passing form such as **Exp** $\rightarrow$ **Env** $\rightarrow$ **Cont** $\rightarrow$ **Answer**. Concurrency further refines the answer type into a thread representation, where **Thread** explicitly carries both a store and a scheduling state. Similarly, type inference extends ordinary type checking by adding substitutions and their refinement to the type-checking process. These patterns show that language evolution in the EOPL family can be understood not only as code modification, but also as systematic changes in evaluator signatures and semantic domains.

The Structure of Interpreters The type signatures of various interpreters can illustrate how different language characteristics influence their structure, as shown in Table 4. For **LET**, **PROC**, and **LETREC**, as well as **SIMPLE-MODULES**, **OPAQUE-TYPES**, and **PROC-MODULES**, the interpreter follows the fundamental type signature **Exp** $\rightarrow$ **Env** $\rightarrow$ **ExpVal**, where expressions are evaluated with an en-

vironment to produce a semantic value. Although `Exp`, `Env`, and `ExpVal` are defined differently across these languages to reflect variations in syntax, scoping and binding, and runtime values, respectively, these differences are abstracted away at the level of the evaluator type, yielding the common type signature.

Table 4. Types of Interpreters

Interpreter	Type Signature
LET, PROC, LETREC SIMPLE-MODULES, OPAQUE-TYPES, PROC-MODULES (CHECKED, INFERRED)	<code>Exp -> Env -> ExpVal</code>
EXPLICIT-REFS, IMPLICIT-REFS	<code>Exp -> Env -> Store -> (ExpVal, Store)</code>
CLASSES, TYPED-00	<code>ClassEnv -> Exp -> Env -> Store -> (ExpVal, Store)</code>
LETREC-CPS ⁽¹⁾ , EXCEPTIONS ^{(1),(2)}	<code>Exp -> Env -> Cont -> ExpVal</code> (1) <code>apply_cont :: Cont -> ExpVal -> ExpVal</code> (2) <code>apply_handler :: ExpVal -> Cont -> ExpVal</code> <code>type Thread = Store -> SchedState -> (ExpVal, Store)</code>
THREADS ⁽³⁾	<code>Exp -> Env -> Cont -> Thread</code> (3) <code>apply_cont :: Cont -> ExpVal -> Thread</code>

In EXPLICIT-REFS and IMPLICIT-REFS, an additional `Store` parameter is introduced whereas the original EOPL textbook retains the same fundamental type signature thereby making stores implicit.

CLASSES and TYPED-00, inheriting the store-passing style interpreter from IMPLICIT-REFS, incorporate `ClassEnv` to support class-based object-oriented programming.

LETREC-CPS and EXCEPTIONS adopt continuation-style interpreters, while THREADS combines this approach with the store-passing style, incorporating `Cont` to express control contexts. EXCEPTIONS introduces two control flows: a normal execution using `apply_cont` and an exceptional flow using `apply_handler`. THREADS supports concurrent execution with an arbitrary number of control flows, executing them one by one in a round-robin schedule. In THREADS, while threads in the original EOPL book are represented with a contract that corresponds to the type `() -> ExpVal`, our approach instead introduces `Store -> SchedState -> (ExpVal, Store)`, explicitly managing both store and scheduling state.

Further details can be found in an extended report [6].

The Structure of Type Checkers The type signatures of various type checkers also illustrate their fundamental structure and variations, as shown in Table 5. The basic type-checking structure follows the signature `Exp -> TyEnv`

Table 5. Types of Type Checkers

Type Checker	Type Signature
CHECKED, SIMPLE-MODULES	<code>Exp -> TyEnv -> Either ErrMsg Type</code>
TYPED-00	<code>StaticClassEnv -> Exp -> TyEnv -> Either ErrMsg Type</code>
INFERRED	<code>Exp -> TyEnv -> Subst -> Either_ ErrMsg (Type, Subst)</code>
OPAQUE-TYPES, PROC-MODULES	<code>Exp -> TyEnv -> Either_ ErrMsg Type</code>

`-> Either ErrMsg Type`, where an expression (`Exp`) and a type environment (`TyEnv`) are given as input, returning either a successfully determined type (`Type`) or an error message (`ErrMsg`) if the expression is ill-typed. This structure is adopted by the type checkers in **CHECKED** and **SIMPLE-MODULES**. **TYPED-00** introduces a static class environment (`StaticClassEnv`) to the basic signature, ensuring that type checking considers class hierarchies.

INFERRED allows programmers to omit type declarations in their programs, with a type inference system automatically determining and filling in the missing types. The system extends the type-checking process by generating fresh type variables for missing types and solving type equations constrained by these variables. The type signature of the type inference in **INFERRED** introduces a type substitution (`Subst`) parameter and replaces `Either` with `Either_`, a variant that incorporates an internal counter state for fresh name generation.

In **OPAQUE-TYPES** and **PROC-MODULES**, the type checkers handle opaque types and procedural modules using essentially the same type signature as in **CHECKED** and **SIMPLE-MODULES**, but using `Either_` for matching bound module variables.

Validating the EOPL Reconstruction We implemented all interpreters and type checkers in Haskell, each comprising approximately 50–364 lines of code, while preserving the behavior of the original object languages presented in EOPL. To validate this preservation, we reused the complete set of original EOPL test cases for each language and confirmed that our Haskell implementations pass them. These tests cover the expected evaluation and type-checking behavior of the corresponding object languages, including representative programs for core language constructs and feature extensions such as procedures, references, exceptions, continuations, threads, modules, and type checking. Passing the original test suites gives evidence that our statically typed and purely functional reconstruction is faithful to the operational behavior of the EOPL implementations, while replacing the surrounding infrastructure—such as parsers, environments, stores, and auxiliary data structures—with typed Haskell modules.

Passing the original EOPL test suites does not guarantee that the two implementations are behaviorally identical. Stronger validation could be obtained by randomly generating additional object-language programs and comparing their

results across the Scheme and Haskell implementations, or by more directly comparing the corresponding implementations and their semantic components.

4 Case Studies

We present three case studies that illustrate progressively broader pedagogical uses of the reconstruction. First, we combine mutable state and exceptions by composing their type-level effects: **Store** for state, and **Cont** with handler continuations for exceptions. Second, we derive an actor-language interpreter from the typed structure of **THREADS**, showing how an existing interpreter can guide the design of a new language. Third, we translate the **CHECKED** interpreter and type checker into Rocq and prove type safety, showing how executable interpreters can serve as a starting point for mechanized reasoning. Together, these studies illustrate a progression from understanding language-feature composition, through language design, to formal verification.

4.1 Combining State and Exceptions by Composing Evaluator Types

This case study focuses on constructing an interpreter for a language that combines mutable state with exception handling. In the EOPL language family, mutable state is introduced by the reference languages, while exception handling is introduced through a continuation-based interpreter. Each feature changes the evaluator architecture in a different way: state requires the evaluator to thread a store, whereas exceptions require the evaluator to make control contexts explicit.

This combination represents a more involved instance of a type-directed reconstruction pattern. A stateful interpreter changes a direct evaluator of type $\text{Exp} \rightarrow \text{Env} \rightarrow \text{ExpVal}$ into a store-passing evaluator of type $\text{Exp} \rightarrow \text{Env} \rightarrow \text{Store} \rightarrow (\text{ExpVal}, \text{Store})$. On the other hand, an exception-handling interpreter introduces continuations and handlers, leading to a continuation-passing evaluator such as $\text{Exp} \rightarrow \text{Env} \rightarrow \text{Cont} \rightarrow \text{Answer}$. To combine the two features, the answer type must also account for mutable state. Thus, the reconstructed evaluator has the form $\text{Exp} \rightarrow \text{Env} \rightarrow \text{Cont} \rightarrow \text{Store} \rightarrow (\text{ExpVal}, \text{Store})$ or, equivalently, by defining $\text{Answer} = \text{Store} \rightarrow (\text{ExpVal}, \text{Store})$, the evaluator can be written as $\text{Exp} \rightarrow \text{Env} \rightarrow \text{Cont} \rightarrow \text{Answer}$.

The important point is that state and exceptions do not merely add new expression forms to the language. They introduce distinct semantic components that must be reflected in the interpreter architecture. The store represents the mutable state of the computation, while the continuation and exception handler represent the normal and exceptional control flows. In the combined interpreter, both normal evaluation and exception propagation must preserve the current store explicitly.

Implementing this interpreter therefore requires coordinated modifications across several semantic components: the abstract syntax is extended with reference operations and exception constructs, **ExpVal** is extended with reference values when locations are explicit, the evaluator is transformed into a store-passing

and continuation-passing form, and the continuation structure is extended with frames for stateful operations and exception handlers. Regression tests from the state and exception languages can be reused to check that each feature still behaves as intended, while additional tests are needed to check their interaction. We therefore added three new tests: one checking that an assignment performed before an exception is raised is visible to the handler, one checking that the handler can update an implicitly allocated variable, and one checking that the raised value is consistent with the store observed by the handler. Both the original regression tests and these three new interaction tests pass. The following is one representative test case, whose expected result is 0.

```
let x = 5 in
  try begin set x = -(x, 1); raise x end
  catch (e) -(e, x)
```

This example illustrates a central benefit of the typed reconstruction. The combination of language features can be understood as the composition of their type-level effects: mutable state contributes an explicit **Store**, while exceptions contribute explicit control contexts through **Cont** and handler continuations. The resulting evaluator type makes the interaction between state and exceptional control flow visible, rather than hiding it in the mutable or dynamically scoped mechanisms of the metalanguage.

4.2 Deriving ACTORS from the Type Structure of THREADS

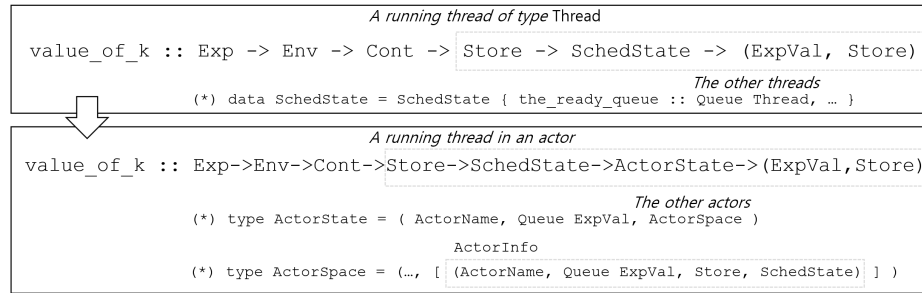


Fig. 3. Designing an Interpreter for ACTORS from the Interpreter for THREADS

The second case study considers the design and implementation of an interpreter for a simple actor-based language, as described in Chapter 4 of *Programming Distributed Computing Systems: A Foundational Approach* [7]. This language extends the lambda calculus with actors. Each actor has a queue for receiving messages from other actors, a local store, and three actor operations:

```

let master =
  proc(self)
    let pingServer =
      proc(self)
        letrec pingServer_self (msg) =
          let PingMsg = 1 in
          let PongMsg = 2 in
          if zero? (-(msg, PingMsg))
          then ready ( proc (masterSelf)
                        send (masterSelf, PongMsg, self))
                    else ready (pingServer_self)
          in ready(pingServer_self)
        in
      in
    letrec master_self (msg) =
      let StartMsg = 0 in
      let PingMsg = 1 in
      let PongMsg = 2 in
      if zero? (-(msg, StartMsg))
      then
        let p = new(pingServer) in
        begin
          send (p, PingMsg, self);
          ready (master_self)
        end
      else if zero? (-(msg, PongMsg)) then print(400)
      else ready (master_self)
      in ready(master_self)
    in let StartMsg = 0 in
      let m = new (master) in
      send (m, StartMsg)

```

Fig. 4. A Ping-pong Example in ACTORS

send, *ready*, and *new*. A Ping-Pong message exchange example written in ACTORS is shown in Figure 4.

ACTORS is not part of the original EOPL language family. We use it to examine whether the typed design space extracted from EOPL can guide the construction of a new interpreter. The starting point is the THREADS interpreter, whose evaluator has the type $\text{Exp} \rightarrow \text{Env} \rightarrow \text{Cont} \rightarrow \text{Thread}$ where $\text{Thread} = \text{Store} \rightarrow \text{SchedState} \rightarrow (\text{ExpVal}, \text{Store})$. This type makes explicit two runtime components of a running thread: the mutable store and the scheduling state. In the original Scheme-based style, such components may be represented using global or mutable host-language state. In our reconstruction, they are explicitly represented in the type of *Thread*.

The actor-based language requires an additional level of runtime structure. A running actor must maintain its own actor name, message queue, local store,

and scheduling state, while the actor system must also keep track of the other actors. Guided by the type structure of `THREADS`, we introduce `ActorState` to maintain the running actor’s name (`ActorName`), its message queue (`Queue ExpVal`), and the surrounding actor space. The actor space consists of actor information records, where each `ActorInfo` contains an actor name, a message queue, a store, and a scheduling state.

Thus, the `ACTORS` interpreter is obtained by refining the runtime structure of `THREADS`. The scheduler state of `THREADS` accounts for the other threads in the ready queue; in `ACTORS`, this idea is lifted to the actor level, where each actor carries its own local queue, store, and scheduler state. As shown in Figure 3, the type of the `THREADS` interpreter directly suggests the additional runtime components required for actors.

Like the `THREADS` interpreter in EOPL, the `ACTORS` interpreter simulates concurrency by interleaving the continuations of multiple running computations. Building on this implementation, we were able to modify the interpreter with the Cloud Haskell (`distributed-haskell`) library [8], allowing actor programs such as the Ping-Pong example to run in a distributed computing environment.

This case study shows that the typed reconstruction goes beyond reimplementing the original EOPL interpreters in Haskell. By making stores and scheduling states explicit, it provides a reusable design pattern for an actor language outside the textbook family. The Ping-Pong example then checks the interaction of message passing, actor creation, and scheduling.

4.3 Mechanizing Type Safety for `CHECKED` in Rocq

The third case study examines whether the typed and purely functional reconstruction can support formal verification. The `CHECKED` language in EOPL consists of a LETREC-style interpreter together with a type checker for the same language. Its expected meta-theoretic property is type safety: if a program is accepted by the type checker, then executing the program with the interpreter should not produce a value inconsistent with the assigned type.

Rather than encoding this soundness relationship using advanced features of Haskell’s type system, we chose to mechanize the type safety argument in Rocq. The Haskell reconstruction was useful for this purpose because the interpreter and type checker were already written in a statically typed and purely functional style. As a result, most of the Haskell definitions could be translated into Rocq definitions directly. The only substantial additional transformation concerns recursion. The `CHECKED` interpreter may define computations that do not terminate, whereas Rocq requires recursive functions to be structurally terminating. We therefore introduced a fuel parameter to the interpreter: evaluation proceeds while fuel remains, and a computation that would otherwise diverge terminates by exhausting the fuel. Apart from this standard fuel-based modeling of potentially nonterminating evaluation, the Rocq development follows the Haskell implementation closely.

The top-level theorem states that type checking is sound with respect to the fuel-based interpreter:

```
Theorem type_sound_value_or_gas : forall (step : nat) (exp : Exp) (ty :
Ty), type_of_program exp = inr ty -> (exists v, value_of_program step exp
= EvalValue v /\ value_has_type v ty) \/ value_of_program step exp =
EvalGasExhausted.
```

In words, if the **CHECKED** type checker assigns type `ty` to an expression `exp`, then the fuel-based interpreter either produces a value of type `ty` or exhausts the available fuel. Fuel exhaustion represents computations that do not terminate within the given bound, including potentially diverging computations.

This case study highlights another benefit of the reconstruction. If the interpreter had remained closer to the original Scheme style, with dynamically typed data representations and mutable or global host-language state, translating it into Rocq would likely have required additional semantic restructuring. In contrast, the Haskell version already makes the relevant semantic domains explicit as algebraic data types and expresses computation through pure functions. This made the Rocq mechanization close to a one-to-one translation of the Haskell artifact. This experience is consistent with prior work such as `seL4` [9] and `hs-to-coq` [10], which use typed functional programs as a basis for mechanized reasoning.

Although this case study mechanizes type safety only for **CHECKED**, the same methodology suggests a path for future work. The *EOPL* family also contains languages with type inference, modules, and typed objects, namely **INFERRED**, the three module languages, and **TYPED-00**. We would like to investigate whether their Haskell reconstructions can be translated into Rocq in a similarly direct manner, and to identify what additional transformations are required beyond the fuel parameter used for **CHECKED**.

5 Discussion and Related Work

5.1 Teaching Experience with the *EOPL* Reconstruction

The reconstruction is currently used in two programming language courses at the author’s university, targeting different academic levels. The earlier part of *EOPL* (8 interpreters and 1 type checker from **LET** through **CHECKED** except **THREADS**) has been taught to third-year undergraduates, while the later part (7 interpreters and 5 type checkers covering **THREADS** and the languages from **INFERRED** to **TYPED-00**) has been taught to first-year master’s students. The corresponding lecture materials and Haskell source code are publicly available.

From Math to Haskell As semantics textbooks typically begin with the basic mathematical techniques needed to describe programming languages, our courses begin with a short introduction to the metalanguage used in the reconstruction. Since most students have little or no prior experience with Haskell, this preparation is essential. We use Lecture 9 of Part II of the Haskell MOOC² to introduce the basic notions needed for the course: types, functions, and programming with

² <https://haskell.mooc.fi/part2#lecture-9-recap-of-part-1>

recursion. The material is particularly useful for self-study because it includes exercises together with tests that allow students to check their solutions.

To keep this prerequisite as small as possible, the reconstructed EOPL deliberately uses only a minimal typed functional vocabulary: algebraic data types, recursive functions, and primitive types such as integers, booleans, and strings. This choice not only makes the semantic components introduced by each language feature explicit, but also lowers the barrier to learning Haskell as the metalanguage.

In particular, the notion of inductive sets introduced in Chapter 1 of EOPL as a mathematical technique for describing syntactic and semantic domains is important, and it is recast using Haskell algebraic data types, whose elements are constructed with data constructors and decomposed by pattern matching. These concepts provide students with the main Haskell tools needed to understand and extend the reconstructed interpreters.

Test-driven and Incremental Development Inspired by agile development practices in software engineering, we structure exercises in our reconstructed EOPL around test-driven and incremental development, allowing students to build interpreters step by step while continuously validating their progress. Although passing the test suite does not guarantee interpreter correctness, it provides immediate feedback on each incremental implementation step, making it particularly useful in an interpreter-based learning approach.

To help students become familiar with both interpreter structure and basic Haskell programming, we provide partially completed projects in which the main interpreter or type-checker functions are left blank³. For example, the `value_of` function of the LET interpreter is already divided into cases for the six expression forms, but the body of each case is removed for students to complete. This exercise encourages students to reconstruct the interpreter themselves. They can then use the same test cases employed for validating our reconstruction to check their implementations.

We additionally encourage an incremental, test-driven construction process. Students first run the test suite with an incomplete interpreter and observe the failures. They then implement only the functionality required by the simplest test cases and gradually increase the number of passing tests. For example, a test containing only a constant expression requires implementing only the `Const_Exp` case for the LET interpreter. A subsequent subtraction test exposes the missing `Diff_Exp` case, which students then implement and verify before proceeding to the next construct. Repeating this process until all tests pass helps students become accustomed to the structure of the LET interpreter and provides a disciplined way to manage the increasing complexity of later interpreters.

Student Feedback from an Undergraduate Course We report feedback from 58 students in the undergraduate programming languages course offered in

³ https://github.com/kwanghoon/Lecture_EOPL_Exercise

Spring 2026. Responses were collected on a five-point Likert scale, where 1 indicated “strongly disagree” and 5 indicated “strongly agree.” Students expressed strong support for the incremental organization of the course—starting from LET and gradually adding language features through a series of interpreters—with an average rating of 4.3. They generally favored learning semantics through executable interpreters rather than through mathematical definitions alone, although some students preferred mathematical descriptions to be used as complementary material. The test-driven and incremental development approach also received an average rating of 4.3. When asked whether the type-directed organization of the interpreters helped them understand their structure, students gave an average rating of 4.2, indicating a positive assessment of the use of types in organizing interpreter designs. In particular, they responded positively to representing syntactic and semantic domains uniformly with algebraic data types.

The main challenge was learning Haskell as the metalanguage. Except for four students, all were encountering Haskell for the first time, and even those four had limited prior experience. Students reported that becoming comfortable with Haskell typically required approximately one to one and a half months, often until shortly before the midterm examination. Feedback on Haskell was therefore mixed. Students identified the initial learning curve as a disadvantage, but also recognized benefits such as concise syntax, pattern matching, algebraic data types, and the way types guide interpreter construction. We believe that our strategy of deliberately restricting the reconstruction to a minimal set of Haskell features may have helped reduce the effort required to learn the metalanguage.

Among the eight object languages covered in the undergraduate course, students reported the greatest difficulty with recursion in LETREC, followed by continuations in LETREC-CPS and explicit references in EXPLICIT-REFS.

Overall, a recurring positive response was that the course provided students, who had previously mainly used programming languages, with a new perspective on how programming languages themselves can be designed and implemented.

From Classroom Use to Language Design One graduate student who used the artifact in both courses won third place in the graduate category of the ACM Student Research Competition at SPLASH 2025 for designing and implementing a multi-tier actor programming language [11]. The project built on the ACTORS case study in Section 4.2 and our prior work on a polymorphic RPC calculus [12]. This provides a concrete example of how the artifact can support students in extending the reconstructed interpreters toward the design and implementation of new programming languages.

5.2 From Explicit Interpreter Structure to Structured Semantics

Directly exposing semantic components in algebraic data types and evaluator signatures, as in our reconstruction, is one straightforward way to structure semantics, but several more systematic approaches have also been developed. Action semantics [13] structures language definitions by decomposing behavior

into reusable semantic actions. Monadic semantics [14,15] represents computational effects such as state, exceptions, input/output, and nondeterminism using monads, allowing interpreters to be organized by their effects rather than by explicit parameter passing. Algebraic effects and handlers [16,17] further separate effect operations from their interpretations, providing a more modular way to describe and combine language features. Garby et al. [18] similarly represent effects explicitly in a purely functional setting and use these representations to derive semantics-preserving compilers by equational reasoning. Although our goal differs, both approaches exploit explicit semantic structure to support systematic reasoning. Interaction trees [19] use a coinductive variant of free monads to represent recursive and effectful computations as executable semantic objects that can subsequently be interpreted by handlers.

Compared with these approaches, our reconstruction exposes semantic components more directly and with less abstraction. In this sense, our interpreters can be viewed as presenting the underlying semantic components in an “unfolded” form, before they are packaged by abstractions such as monads. We view this simplicity as pedagogically useful: students can directly observe how each language extension changes the evaluator type and introduces corresponding semantic components, such as stores, continuations, and scheduling states. After encountering these components explicitly, more structured techniques can be introduced as abstractions over them. In this sense, the reconstruction provides a concrete bridge from pedagogical definitional interpreters to more systematic approaches to structuring executable and formal semantics.

6 Conclusion and Future Work

This paper reconstructed the family of EOPL-style definitional interpreters in Haskell for pedagogical use, preserving the original object languages, examples, and tests while making semantic components explicit through algebraic data types, evaluator signatures, and purely functional state passing. Across 15 interpreters and 6 type checkers, the reconstruction shows students how language extensions correspond to systematic changes in semantic domains and evaluator types. Its use in undergraduate and graduate programming language courses and the three case studies illustrate a progression from understanding language features, through extending and designing interpreters, to formal reasoning about programming languages.

Future work includes further evaluating the pedagogical effectiveness of the reconstruction through more systematic classroom studies, extending the approach to additional language features beyond EOPL, and scaling the Rocq mechanization to the remaining typed languages.

Acknowledgments. This work was supported by the IITP grant (IITP-2023-RS-2023-00256629), the NRF of Korea grant funded by the Korea government (RS-2025-24523420), and the KISA Information Security College Support Project.

Disclosure of Interests. The author has no competing interests to declare.

References

1. Winskel, G.: The Formal Semantics of Programming Languages: An Introduction. MIT Press, Cambridge, MA, USA (1993)
2. Pierce, B.C., de Amorim, A.A., Casinghino, C., Gaboardi, M., Greenberg, M., Hrițcu, C., Sjöberg, V., Tolmach, A., Yorgey, B.: Programming Language Foundations, Software Foundations, vol. 2. Electronic textbook (2026), version 7.0, <http://softwarefoundations.cis.upenn.edu>
3. Wadler, P., Kokke, W., Siek, J.G.: Programming Language Foundations in Agda (Aug 2022), <https://plfa.inf.ed.ac.uk/22.08/>
4. Friedman, D.P., Wand, M.: Essentials of Programming Languages. The MIT Press, Cambridge, Massachusetts, third edn. (2008)
5. Burstall, R.M., MacQueen, D.B., Sannella, D.T.: Hope: An experimental applicative language. In: Proceedings of the 1980 ACM Conference on LISP and Functional Programming. p. 136–143. LFP '80, Association for Computing Machinery, New York, NY, USA (1980). <https://doi.org/10.1145/800087.802799>.
6. Choi, K.: Statically typed purely functional definitional interpreters for *Essentials of Programming Languages*, https://github.com/kwanghoon/Lecture_EOPL, [Online; accessed September 1, 2026]
7. Varela, C.A.: Programming Distributed Computing Systems: A Foundational Approach. The MIT Press, Cambridge, Massachusetts (2013)
8. Epstein, J., Black, A., Peyton Jones, S.: Towards Haskell in the cloud. In: Haskell Symposium (September 2011), <https://www.microsoft.com/en-us/research/publication/towards-haskell-cloud/>
9. Klein, G., Elphinstone, K., Heiser, G., Andronick, J., Cock, D., Derrin, P., Elkaduwe, D., Engelhardt, K., Kolanski, R., Norrish, M., Sewell, T., Tuch, H., Winwood, S.: sel4: formal verification of an os kernel. In: Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles. p. 207–220. SOSP '09, Association for Computing Machinery, New York, NY, USA (2009). <https://doi.org/10.1145/1629575.1629596>.
10. Spector-Zabusky, A., Breitner, J., Rizkallah, C., Weirich, S.: Total haskell is reasonable coq. In: Proceedings of the 7th ACM SIGPLAN International Conference on Certified Programs and Proofs. p. 14–27. CPP 2018, Association for Computing Machinery, New York, NY, USA (2018). <https://doi.org/10.1145/3167092>.
11. Lee, H.: On a multitier actor programming language. In: Companion Proceedings of the 2025 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity. p. 40–42. SPLASH Companion '25, Association for Computing Machinery, New York, NY, USA (2025). <https://doi.org/10.1145/3758316.3763252>.
12. Choi, K., Cheney, J., Fowler, S., Lindley, S.: A polymorphic rpc calculus. Science of Computer Programming **197**, 102499 (2020). <https://doi.org/10.1016/j.scico.2020.102499>.
13. Mosses, P.D.: Theory and practice of action semantics. Tech. Rep. RS-96-53, University of Aarhus, Aarhus, DK (December 1996)
14. Liang, S., Hudak, P., Jones, M.: Monad transformers and modular interpreters. In: Proceedings of the 22nd ACM SIGPLAN-SIGACT symposium on Principles of programming languages. p. 333–343. POPL '95, Association for Computing Machinery, New York, NY, USA (1995). <https://doi.org/10.1145/199448.199528>.
15. Wadler, P.: The essence of functional programming. In: Proceedings of the 19th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages.

- p. 1–14. POPL '92, Association for Computing Machinery, New York, NY, USA (1992). <https://doi.org/10.1145/143165.143169>.
16. Plotkin, G., Pretnar, M.: Handlers of algebraic effects. In: Proceedings of the 18th European Symposium on Programming Languages and Systems: Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2009. p. 80–94. ESOP '09, Springer-Verlag, Berlin, Heidelberg (2009). https://doi.org/10.1007/978-3-642-00590-9_7.
 17. Pretnar, M.: An introduction to algebraic effects and handlers. invited tutorial paper. Electron. Notes Theor. Comput. Sci. **319**(C), 19–35 (Dec 2015). <https://doi.org/10.1016/j.entcs.2015.12.003>.
 18. Garby, Z., Hutton, G., Bahr, P.: Calculating compilers effectively (functional pearl). In: Proceedings of the 17th ACM SIGPLAN International Haskell Symposium. p. 109–119. Haskell 2024, Association for Computing Machinery, New York, NY, USA (2024). <https://doi.org/10.1145/3677999.3678283>.
 19. Xia, L.y., Zakowski, Y., He, P., Hur, C.K., Malecha, G., Pierce, B.C., Zdancewic, S.: Interaction trees: representing recursive and impure programs in coq. Proc. ACM Program. Lang. **4**(POPL) (Dec 2019). <https://doi.org/10.1145/3371119>.
 20. Lim, J., Kim, G., Shin, S., Choi, K., Kim, I.: Parser generators sharing lr automaton generators and accepting general purpose programming language-based specifications. Journal of KIISE **40**(1), 52–60 (2020)
 21. Aho, A.V., Lam, M.S., Sethi, R., Ullman, J.D.: Compilers: Principles, Techniques, and Tools. Addison-Wesley, Boston, MA, USA, 2nd edn. (2006)
 22. Andy Gill and Simon Marlow: happy: Happy is a parser generator for Haskell (2009), <https://www.haskell.org/happy/>, [Online; accessed February 18, 2025]

A A Parsing System and Abstract Syntax Trees (Exp)

EOPL provides the SLLGEN tool [4], which is based on LL parsing and written directly in Scheme, while we used the YAPB tool [20], which is based on LR parsing and implemented in Haskell. A key advantage of using a parsing library that allows both the parser and the interpreter to be written in the same implementation language is that it eliminates the need for a separate language (e.g., YACC [21], Happy [22]) specifically for parser construction.

The lexer and parser for the LET language in Figure 5 define its syntactic structure and tokenization process. The lexer (`lexerSpec`) recognizes keywords (`if`, `then`, `else`, `let`, `in`), symbols (`-`, `(`, `)`, `..`, `=`), numbers, and identifiers, converting them into corresponding tokens. The parser (`parserSpec`) then defines the grammar rules for expressions, supporting constants, negation, arithmetic operations (`Diff_Exp`), conditional expressions (`If_Exp`), variable references (`Var_Exp`), and let bindings (`Let_Exp`). By combining these components, the system transforms raw text input into an abstract syntax tree (AST) that can be interpreted or further processed.

B Type Definitions in the Interpreters

B.1 Environment Types

Table 6 summarizes the environment (`Env`) definitions used in various interpreters, highlighting how they evolve to accommodate different language fea-

```

lexerSpec :: LexerSpec Token IO ()
lexerSpec = LexerSpec
{
  lexerSpecList =
    [ ("[ \t\n]" , \_ -> return Nothing),
      ("[0-9]+" , \_ -> return (Just INTEGER_NUMBER)),
      ("\\-" , \_ -> return (Just SUB)),
      ("\\(" , \_ -> return (Just OPEN_PAREN)),
      ("\\)" , \_ -> return (Just CLOSE_PAREN)),
      ("\\," , \_ -> return (Just COMMA)),
      ("zero\\?" , \_ -> return (Just ISZERO)),
      ("if" , \_ -> return (Just IF)),
      ("then" , \_ -> return (Just THEN)),
      ("else" , \_ -> return (Just ELSE)),
      ("let" , \_ -> return (Just LET)),
      ("in" , \_ -> return (Just IN)),
      ("\\=" , \_ -> return (Just EQ)),
      ("[a-zA-Z][a-zA-Z0-9]*" , \_ -> return (Just IDENTIFIER))
    ]
}

parserSpec :: ParserSpec Token Exp IO ()
parserSpec = ParserSpec
{
  parserSpecList =
    [
      rule "Expression' -> Expression"
        (\rhs -> return $ get rhs 1),
      rule "Expression -> integer_number"
        (\rhs -> return $ Const_Exp (read (getText rhs 1) :: Int)),
      rule "Expression -> - integer_number"
        (\rhs -> return $ Const_Exp (-(read (getText rhs 2) :: Int))),
      rule "Expression -> - ( Expression , Expression )"
        (\rhs -> return $ Diff_Exp (get rhs 3) (get rhs 5)),
      rule "Expression -> zero? ( Expression )"
        (\rhs -> return $ IsZero_Exp (get rhs 3)),
      rule "Expression -> if Expression then Expression else Expression"
        (\rhs -> return $ If_Exp (get rhs 2) (get rhs 4) (get rhs 6)),
      rule "Expression -> identifier"
        (\rhs -> return $ Var_Exp (getText rhs 1)),
      rule "Expression -> let identifier = Expression in Expression"
        (\rhs -> return $ Let_Exp (getText rhs 2) (get rhs 4) (get rhs 6))
    ]
}

```

Fig. 5. Lexer and Parser Specifications for LET (An Excerpt)

tures. In LET and PROC, the environment consists of an empty environment (`Empty_env`) and an extension mapping an identifier to an expression value (`Extend_env`). LETREC extends this structure by adding recursive bindings with `Extend_env_rec`. By modifying `apply_env`, to create a procedure value (also called a closure) at the time of a call rather than at the time of declaration, this environment structure enables a purely functional implementation of recursive procedures. All subsequent languages after LETREC in Figure 1 adopt this structure as is or with modification to support mutually recursive definitions. EXPLICIT-REFS and IMPLICIT-REFS introduce references, storing either expressed values, allowing programmers to handle locations like C pointers, or locations, interpreting variables as references like in Java. The module languages, SIMPLE-MODULES, OPAQUE-TYPES, and PROC-MODULES further extend the environment with `Extend_env_with_module`, enabling module-based scoping. Finally, CLASSES and TYPED-OO incorporate class-based features with `Extend_env_with_self_and_super`, to support the `self` and `super` keywords, enabling class hierarchies and method resolution.

Thus these variations reflect how different language features influence the design of an interpreter's environment.

Note that `apply_procedure` is a runtime library that retrieves the saved environment from a procedure value (closure), extends it with a binding between the procedure's parameter variable and the provided argument, and evaluates the body within the extended environment using `value_of`.

```

apply_procedure :: Proc -> ExpVal -> ExpVal
apply_procedure (Procedure var body env) arg =
  value_of body (extend_env var arg env)

```

B.2 Expressed Value Type Definitions

Table 7 presents different variations of the `ExpVal` (Expressed Values) definition used in various interpreters. The LET interpreter supports only numerical and boolean values, represented by `Num_Val` and `Bool_Val`. The PROC, LETREC, IMPLICIT-REFS, LETREC-CPS, CHECKED, INFERRED, SIMPLE-MODULES, OPAQUE-TYPES, and PROC-MODULES interpreters extend this by adding `Proc_Val`, which represents procedures. The EXPLICIT-REFS interpreter further introduces `Ref_Val`, which represents reference values to memory locations. The EXCEPTIONS interpreter adds support for lists through `List_Val`, enabling structured data manipulation. The THREADS interpreter builds upon this by incorporating concurrency-related values, including `Mutex_Val` for mutual exclusion mechanisms and `Queue_Val` for thread queues. Finally, the CLASSES and TYPED-OO interpreters introduce object-oriented features with `Object_Val` to represent objects and `Uninitialized_Val` for uninitialized fields, supporting class-based programming paradigms. This progression of `ExpVal` definitions reflects how different interpreters extend the expressiveness of values to accommodate various computational paradigms.

Note that the inclusion of `Queue_Val` as one of the variants of expressed values is somewhat inconsistent, as the THREADS interpreter never produces

Table 6. Environment Type Definitions in Different Interpreters

Interpreter	Environment Definition
LET, PROC	<pre>data Env = Empty_env Extend_env Ident ExpVal Env</pre>
LETREC, LETREC-CPS, EXCEPTIONS (CHECKED, INFERRED)	<pre>data Env = Empty_env Extend_env Ident ExpVal Env Extend_env_rec Ident Ident Exp Env</pre>
EXPLICIT-REFS	<pre>data Env = Empty_env Extend_env Ident ExpVal Env Extend_env_rec [(Ident, Ident, Exp)] Env</pre>
IMPLICIT-REFS, THREADS	<pre>data Env = Empty_env Extend_env Ident Location Env Extend_env_rec [(Ident, Ident, Exp)] Env</pre>
SIMPLE-MODULES, OPAQUE-TYPES, PROC-MODULES	<pre>data Env = Empty_env Extend_env Ident ExpVal Env Extend_env_rec Ident Ident Exp Env Extend_env_with_module Ident TypedModule Env</pre>
CLASSES, TYPED-OO	<pre>data Env = Empty_env Extend_env [Ident] [Location] Env Extend_env_rec [(Ident,[Ident],Exp)] Env Extend_env_with_self_and_super Object Ident Env</pre>

queue values from evaluating expressions in the language. However, this inclusion stems from the strategy used to implement mutex queues. A mutex consists of a boolean value indicating whether the critical region it protects is currently in use and a queue for threads waiting when the critical region is occupied. While an expression may evaluate to a mutex, the queue itself is not directly manipulated by programs. However, mutex queues are stored in the same store that holds expressed values, and the functions `newref :: Store -> ExpVal -> (Location, Store)`, `setref :: Store -> Location -> ExpVal -> Store`, and `deref :: Store -> Location -> ExpVal` is used uniformly to create, store, and retrieve both expressed values and mutex queues in the store. As a result, `Queue_Val` was included in `ExpVal` to maintain consistency in storage and retrieval operations.

Also note that a special expressed value `Uninitialized_Val` should have been introduced to indicate uninitialized state in `CLASSES` and `TYPED-OO`.

B.3 Type Definition of Store in EXPLICIT-REFS and IMPLICIT-REFS

Note that **Store** is the type for a mapping of locations to expressed values with a counter to generate fresh locations as $(\text{Location}, [(\text{Location}, \text{ExpVal})])$.

B.4 The Structure of Interpreters

The type signatures of various interpreters can illustrate how different language characteristics influence their structure, as shown in Table 4. For **LET**, **PROC**, and **LETREC**, as well as **SIMPLE-MODULES**, **OPAQUE-TYPES**, and **PROC-MODULES**, the interpreter follows the fundamental type signature $\text{Exp} \rightarrow \text{Env} \rightarrow \text{ExpVal}$, where expressions are evaluated with an environment to produce a semantic value. While this type signature abstracts over differences in environment structures, the interpreters for these language features are distinguished by their specific **Env** definitions, which capture the variations in scoping and binding mechanisms, as summarized by Table 6 in Appendix.

In **EXPLICIT-REFS** and **IMPLICIT-REFS**, an additional **Store** parameter is introduced whereas the original EOPL textbook retains the same fundamental type signature thereby making stores implicit. This type signature reflects the store-passing style, where evaluation produces a value while also updating the store, representing mutable state. Note that **Store** is the type for a mapping of locations to expressed values. The **Store** type deserves to appear in the signature because stores are fundamental elements. The difference between the two languages can be explained by their differing data types for expressed values. In particular, the data type **ExpVal** for **EXPLICIT-REFS** includes a variant **Ref_Val** $\{\text{expva_loc} :: \text{Location}\}$, allowing programs to directly use locations, similar to C, whereas the data type for **IMPLICIT-REFS** does not, as in Java. The type definitions of expressed values are available in Table 7 in Appendix.

CLASSES and **TYPED-OO**, inheriting the store-passing style interpreter from **IMPLICIT-REFS**, incorporates **ClassEnv** to support class-based object-oriented programming. Their expressed values are ones that extend those of **IMPLICIT-REFS** with object values and the special value denoting uninitialized fields.

LETREC-CPS and **EXCEPTIONS** adopt continuation-style interpreters, while **THREADS** combines this approach with the store-passing style, incorporating **Cont** to express control contexts. They use **apply_cont** to apply a continuation to a computed value. **EXCEPTIONS** introduces two control flows: a normal execution using **apply_cont** and an exceptional flow using **apply_handler**, which handles exceptions (of type **ExpVal**) by modifying the control flow. **THREADS** supports concurrent execution with an arbitrary number of control flows, executing them one by one in a round robin schedule. In **THREADS**, while threads in the original EOPL book are represented with a contract that corresponds to the type $(\) \rightarrow \text{ExpVal}$, our approach instead introduces **Store** $\rightarrow$ **SchedState** $\rightarrow (\text{ExpVal}, \text{Store})$, explicitly managing both store and scheduling state. This shift highlights the advantages of using static types, which helped identify an implicit definition of threads in the original EOPL book.

Also, using static types, we identified an imprecise type definition of expressed values in which the queue type had to be included as a variant. This was due to the structure of storing the runtime-managed thread queue in the store, which cannot be directly derived from evaluating **THREADS** program expressions but appears in **ExpVal** (See `Queue_Val {expval_queue :: Queue Thread}` in Table 7 of the Appendix).

The explicit type signature of **THREADS** for shared-memory systems proved useful in designing an interpreter for a new programming language, **ACTORS**, for distributed systems, as will be discussed in Section 4.

C Type Definitions in the Type Checkers

C.1 Type Environment Types

Table 8 presents different variations of the **TyEnv** (Type Environment) used in the type checker across different modules. Each row corresponds to a specific category of the type checker, with the left column indicating the category and the right column showing the corresponding **TyEnv** definition. In **CHECKED** and **INFERRED**, the type environment consists of an empty environment (`Empty_tyenv`) and an extension mechanism (`Extend_tyenv`) to associate identifiers with types. In **SIMPLE-MODULES**, this extends **TyEnv** by adding `Extend_tyenv_with_module`, allowing modules to be included in the type environment. In **OPAQUE-TYPES** and **PROC-MODULES**, further extends the type environment with `Extend_tyenv_with_type`, which introduces named types. In **TYPED-OO**, `Extend_tyenv_with_self_and_super` is introduced, which supports self and supertype relationships, useful for class-based type systems. This table illustrates how the **TyEnv** structure evolves to accommodate various features in different type checker modules.

C.2 Type and Interface Definitions

Table 9 presents the different variations of the **Type** definition used in various type checkers. Each row corresponds to a specific category of the type checker, with the left column indicating the category and the right column showing the corresponding **Type** definition. In **CHECKED** and **SIMPLE-MODULES**, the basic type system consists of primitive types `TyInt` and `TyBool`, along with function types represented by `TyFun`. In **INFERRED**, this extends the type system by adding `TyVar`, which represents type variables identified by integers, enabling type inference. **OPAQUE-TYPES** and **PROC-MODULES** introduce `TyName` and `TyQualified`, which allow named and qualified types, respectively, enhancing modularity and abstraction. **TYPED-OO** further expand the type system with `TyVoid` (representing void types), `TyFun` supporting lists of argument types, `TyClass` for class-based types, and `TyListOf` for list types. This table illustrates how the type system evolves to accommodate different levels of abstraction and modularity in various type checking environments.

Table 10 outlines the variations of the **Interface** definition across different type checkers. The left column specifies the type checker category, while the right column contains the corresponding **Interface** definition. In **SIMPLE-MODULES** and **OPAQUE-TYPES**, we define a basic module interface with **SimpleIfc**, which consists of a list of declarations. **PROC-MODULES** extends the interface definition by introducing **ProcIfc**, which allows for procedural module interfaces. This enables structured interaction between different modules, supporting hierarchical compositions of interfaces. This table highlights the evolution of interface definitions, demonstrating how more advanced module composition features are incorporated in different type checkers.

C.3 The Structure of Type Checkers

The type signatures of various type checkers also illustrate their fundamental structure and variations, as shown in Table 5. The basic type-checking structure follows the signature `Exp -> TyEnv -> Either ErrMsg Type`, where an expression (**Exp**) and a type environment (**TyEnv**) are given as input, returning either a successfully inferred type (**Type**) or an error message (**ErrMsg**) if the expression is ill-typed. This structure is adopted by the type checkers in **CHECKED** and **SIMPLE-MODULES**. **TYPED-OO** introduces a static class environment (**StaticClassEnv**) to the basic signature, ensuring that type checking considers class hierarchies.

INFERRED allows programmers to omit type declarations in their programs, with a type inference system automatically determining and filling in the missing types. The system extends the type-checking process by generating fresh type variables for missing types and solving type equations constrained by these variables. The type signature of the type inference in **INFERRED** introduces a type substitution (**Subst**) parameter and replaces **Either** with **Either_**, a variant that incorporates an internal counter state. The first change reflects how type inference proceeds by analyzing the structure of **Exp**, gradually refining the input type substitution to include more constraints on type variables as it progresses. For example, in the expression `Call_Exp rator rand`, after inferring the types **ty1** and **ty2** for **rator** and **rand**, respectively, the system unifies (**ty1**, **ty2** -> **fresh_tyvar**). At this point, an internal count is incremented to generate the fresh type variable, and upon successful unification, the system returns it.

The second change accounts for both side effects: determining whether type inference succeeds or fails (**ExceptT e m a**) and managing a counter state for generating new type variable names during the inference process (**State Integer** for **m**). **Either_ e a** is defined as `type Either_ e a = ExceptT e m a` which represents a monadic computation that combines exception handling (**ExceptT**) and state management (**State Integer**) [15] [14]. This definition extends error handling with stateful computations, enabling fresh name generation, while maintaining nearly the same structure as when **Either** was used. We deliberately designed a custom-defined data type, **Either_**, to resemble **Either**, which is a builtin algebraic data type defined as `data Either a b = Left a | Right b`,

and represents computations that may result in either success or failure. An advantage of using `Either` and `Either_` is that they explicitly convey the scope of side effects in the alternative and stateful computations within the type checker implementations.

In `OPAQUE-TYPES` and `PROC-MODULES`, the type checkers handle opaque types and procedural modules using essentially the same type signature as in `CHECKED` and `SIMPLE-MODULES`, namely `Exp -> TyEnv -> Either_ ErrMsg Type`. However, they rely on `Either_`,... rather than `Either`,... In `OPAQUE-TYPES`, fresh module names must be generated to avoid conflicts when opaque types are compared with qualified types, thereby keeping each interface clearly separated from its implementation. In `PROC-MODULES`, fresh module names are also needed for argument modules when checking the compatibility of module procedure interfaces. These requirements for fresh names are reflected in the use of the variant type with `Either_`.

Note that since `Store` and `Subst` are essential components of programming languages, they deserve to be explicitly included in the type signature. In contrast, the counter state for generating new names is not a major component, so its existence was merely highlighted through the `Either_` type, thus making it implicit.

Interpreter	Expressed Value Definition
LET	<pre> data ExpVal = Num_Val {expval_num :: Int} Bool_Val {expval_bool :: Bool} </pre>
PROC, IMPLICIT-REFS, LETREC-CPS, INFERRED, SIMPLE-MODULES, OPAQUE-TYPES, PROC-MODULES	<pre> LETREC, CHECKED, data ExpVal = Num_Val {expval_num :: Int} Bool_Val {expval_bool :: Bool} Proc_Val {expval_proc :: Proc} </pre>
EXPLICIT-REFS	<pre> data ExpVal = Num_Val {expval_num :: Int} Bool_Val {expval_bool :: Bool} Proc_Val {expval_proc :: Proc} Ref_Val {expval_loc :: Location} </pre>
EXCEPTIONS	<pre> data ExpVal = Num_Val {expval_num :: Int} Bool_Val {expval_bool :: Bool} Proc_Val {expval_proc :: Proc} List_Val {expval_list :: [ExpVal]} </pre>
THREADS	<pre> data ExpVal = Num_Val {expval_num :: Int} Bool_Val {expval_bool :: Bool} Proc_Val {expval_proc :: Proc} List_Val {expval_list :: [ExpVal]} Mutex_Val {expval_mutex :: Mutex } Queue_Val {expval_queue :: Queue Thread} </pre>
CLASSES, TYPED-00	<pre> data ExpVal = Num_Val {expval_num :: Int} Bool_Val {expval_bool :: Bool} Proc_Val {expval_proc :: Proc} List_Val {expval_list :: [ExpVal]} Object_Val {expval_obj :: Object} Uninitialized_Val </pre>

Table 7. Different Variants of Expressed Values in the Interpreters

Type Checker	Type Environment Definition
CHECKED, INFERRED	<pre>data TyEnv = Empty_tyenv Extend_tyenv Ident Type TyEnv</pre>
SIMPLE-MODULES	<pre>data TyEnv = Empty_tyenv Extend_tyenv Ident Type TyEnv Extend_tyenv_with_module Ident Interface TyEnv</pre>
OPAQUE-TYPES, PROC-MODULES	<pre>data TyEnv = Empty_tyenv Extend_tyenv Ident Type TyEnv Extend_tyenv_with_module Ident Interface TyEnv Extend_tyenv_with_type Ident Type TyEnv</pre>
TYPED-00	<pre>data TyEnv = Empty_tyenv Extend_tyenv Ident Type TyEnv Extend_tyenv_with_self_and_super Type Ident TyEnv</pre>

Table 8. Different Variants of TyEnv in the Type Checker

Type Checker	Type Definition
CHECKED, SIMPLE-MODULES	<pre>data Type = TyInt TyBool TyFun Type Type</pre>
INFERRED	<pre>data Type = TyInt TyBool TyFun Type Type TyVar Integer</pre>
OPAQUE-TYPES, PROC-MODULES	<pre>data Type = TyInt TyBool TyFun Type Type TyName Ident TyQualified Ident Ident</pre>
TYPED-00	<pre>data Type = TyInt TyBool TyVoid TyFun [Type] Type TyClass Ident TyListOf Type</pre>

Table 9. Different Variants of Type in the Type Checker

Type Checker	Interface Definition
SIMPLE-MODULES, OPAQUE-TYPES	<pre>data Interface = SimpleIface [Declaration]</pre>
PROC-MODULES	<pre>data Interface = SimpleIface [Declaration] ProcIface Ident Interface Interface</pre>

Table 10. Different Variants of Interface in the Type Checker